

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.23

DATABASE EXCEPTIONS

 Copy as Markdown

Database exceptions

Serverpod wraps database failures in exceptions that are subtypes of `DatabaseException`. This gives you one common type to catch for database failures and more specific types when you want to handle a known failure, such as a unique constraint violation or a single-row operation that matched no row.

```
try {
  await Company.db.updateById(
    session,
    companyId,
    columnValues: (t) => [t.name('New name')],
  );
} on DatabaseUnexpectedResultException {
  // No row with the provided id was updated.
} on DatabaseQueryException catch (e) {
  session.log(
    'Database query failed with code ${e.code}',
    exception: e,
  );
} on DatabaseException catch (e) {
  session.log('Database operation failed', exception: e);
}
```

When a database exception is not caught inside an endpoint, it follows Serverpod's normal endpoint exception handling and is logged as an uncaught server exception. Serverpod does not serialize database exception details and send them to the app. They stay server-side in the logs. See [Error handling and exceptions](#) for how uncaught exceptions reach the app.

Exception types



Exception	Subtype of	When it is thrown
<code>DatabaseException</code>	<code>Exception</code>	The base type for database exceptions. Catch this when you want one handler for any database failure.
<code>DatabaseUnexpectedResultException</code>	<code>DatabaseException</code>	A single-row operation did not affect exactly one row: <code>insertRow</code> returned a different number of rows, or <code>updateRow</code> , <code>updateById</code> , or <code>deleteRow</code> matched no row. <code>upsertRow</code> returns <code>null</code> instead when the existing row doesn't match its <code>updateWhere</code> expression.
<code>DatabaseQueryException</code>	<code>DatabaseException</code>	The database rejected a query. Carries the adapter's error details, see below.
<code>DatabaseUniqueViolationException</code>	<code>DatabaseQueryException</code>	A write violated a unique index or primary key.
<code>DatabaseForeignKeyViolationException</code>	<code>DatabaseQueryException</code>	A write violated a foreign key constraint, including at commit for deferrable constraints .
<code>SQLiteDatabaseLockedException</code>	<code>DatabaseQueryException</code>	SQLite could not acquire a lock, typically because a query ran without the <code>transaction</code> of an already active transaction, or two transactions ran concurrently.
<code>SQLiteMigrationForeignKeyViolationException</code>	<code>DatabaseException</code>	A SQLite foreign key integrity check after migrations found one or more violating rows.

Query exception details

The `DatabaseQueryException` type and its subclasses expose optional fields from the underlying database adapter. These fields are useful for logging and for handling known database errors:

- `code`
- `detail`
- `hint`
- `tableName`
- `columnName`
- `constraintName`
- `position`

These values are database-adapter details, so write defensive code that handles `null` values. PostgreSQL fills in the violated constraint name. SQLite does not. To react to a specific failure, prefer the typed subclass over inspecting the fields:

```
try {
    await Company.db.insertRow(session, company);
} on DatabaseUniqueViolationException {
    // Handle a duplicate company name here.
    return;
}
```

Other query failures keep their original type and stack trace, so a broader `on DatabaseQueryException` handler can log them.

Operation exceptions

`DatabaseUnexpectedResultException` describes cases where Serverpod expected one row to be affected but the database result did not match that expectation. For example, `updateById` throws it when no row exists for the id you pass in.

Batch and filtered operations have their own documented behavior. Some methods, such as `updateWhere` or `deleteWhere`, can validly affect zero rows and return an empty list instead of throwing.

SQLite foreign key checks

When using SQLite, Serverpod runs a foreign key integrity check after applying migrations in the development run mode. If SQLite reports invalid foreign key references, Serverpod throws `SQLiteMigrationForeignKeyViolationException`. Its `violations` field holds the rows returned by SQLite's `PRAGMA foreign_key_check`.